SOFTWARE QUALITY ASSURANCE · PROJECT REPORT

RentaTool
Testing Guide & Report

Detailed web, API, and Flutter test cases, recorded outcomes, and repeatable run commands.

Project: RentaTool rental platform

Source documents: NFR report, test plan, run guide, and Flutter test report

Report date: 7 October 2026

Coverage: Web, API, Flutter mobile, accessibility, load, security, reliability, and recovery

Results and limits follow the saved logs. No unverified case is reported as passed.

Detailed NFR Test Report

SE3090 Non-functional testing report

Scope and real-system inventory

This report covers only non-functional testing. Functional unit, widget, and routine E2E suites are out of scope and are not counted as NFR evidence.

Flutter unit/widget, functional E2E, and catalog API integration cases are documented separately in `FLUTTER_TEST_REPORT.md`; those results are not counted in the NFR summary below.

- React app: `src/web` (React 18/Vite); login `/login`; protected operations, catalog, bookings, identity/KYC, claims, escrow and arbitration routes. User journeys found include login, equipment browsing/management, KYC, booking/handover, claims and escrow.
- ASP.NET Core API: `src/backend/Host/RentaTool.API`; modular Identity, Catalog, Booking and Escrow APIs. The auth endpoints are `POST /api/v1/auth/login` and `/register`; public reads include `GET /api/v1/equipment`; admin/identity endpoints include `GET /api/v1/users` (Admin role), `GET /api/v1/users/{id}` (authenticated, self/Admin).
- PostgreSQL: local Windows service `postgresql-x64-18`, configured by the API connection string. Credentials are not reproduced here.
- Flutter app: sibling repo `../RentaTool-mobile`; login/register/KYC, catalog, booking/handover and escrow/claims screens. Connected physical device: Samsung SM-E556B (Android 16, API 36; ADB serial `R9ZXA0206LN`).
- Agentic AI: `src/ai_service/main.py`, FastAPI health and `/api/v1/ai/*` routes. Python runtime was unavailable, so no live AI service test was possible.

The API was started in Development at `http://127.0.0.1:5000`; Vite served the actual React app at `http://127.0.0.1:3000`. API startup ran the existing `EnsureCreated`/schema compatibility and seed checks against the configured local PostgreSQL database. Test requests were read-only apart from an invalid login; no business records were created or changed.

Summary

ID	NFR Type	Component	Tool	Result
NFR-PERF-001	Performance	Web	Playwright	PASS, 230.60 ms DOMContentLoaded on latest local run
NFR-PERF-002	Performance	API	curl	PASS, observed response timings
NFR-PERF-003	Performance	Flutter	Flutter profile/integration tooling	NOT RUN, frame metrics not collected
NFR-LOAD-001	Load	API	k6	PASS, 280 requests, zero errors
NFR-STRESS-001	Stress	API	k6	PASS, exploratory read-only profile through 30 configured VUs; no failure observed
NFR-SEC-001	Security	API	curl	PASS, tested probes only
NFR-SEC-002	Security	API	curl	PASS after fix; allowlisted origin accepted, untrusted origin denied

NFR-SEC-003	Security	Web/API	OWASP ZAP	BLOCKED, installer blocked by Windows security
NFR-USE-001	Usability	Web	Playwright	PASS, targeted login checks
NFR-USE-002	Usability	Flutter	Flutter integration_test	PARTIAL; semantics finder narrowed, device retest blocked
NFR-A11Y-001	Accessibility	Web	Lighthouse/axe	PASS, automated audits; 0 axe violations; Lighthouse accessibility 1.00
NFR-A11Y-002	Accessibility	Web	Playwright	PASS, limited names/focus check
NFR-COMP-001	Compatibility	Web	Playwright Chromium	PASS, three viewport sizes
NFR-COMP-002	Compatibility	Web	Playwright	PASS, login check on Chromium, Firefox, WebKit
NFR-COMP-003	Compatibility	Flutter	Flutter integration_test	PASS, limited to Android 16 phone/login screen
NFR-COMP-004	Compatibility	Web	Playwright Chromium	PASS, login route/control render
NFR-REL-001	Reliability	API	PowerShell + curl	PASS, 20/20 reads succeeded
NFR-REL-002	Reliability	Flutter	Flutter integration_test	NOT RUN, repeated/offline scenario not executed
NFR-REC-001	Recovery	API	PowerShell process restart + HTTP probes	PASS (limited API scope), 5 checks passed; health recovered in 1.59 seconds

The exploratory k6 stress profile was read-only and reached a measured maximum of 29 active VUs out of 30 configured. It is not a capacity certification because no expected production load or SLA is specified.

Tools used

Tool	NFR use	Status/evidence
Playwright Test + Chromium/Firefox/WebKit	Web usability, responsive layout, axe scan, compatibility and navigation timing assertion	Executed; three-browser checks passed; screenshots/output under `src/web/test-results/`, `src/web/nfr-test-results/`, `evidence/web/`, and `evidence/axe-login-*.png`
curl.exe	Live API response timing, authentication/authorization and CORS probes	Executed; see `evidence/api-probes.txt` and `evidence/security-probes.txt`
k6	Read-only API load and exploratory stress profiles	Load 1→3→5 VUs and stress 5→10→20→30 VUs executed; see `evidence/k6-summary.json` and `evidence/k6-stress-summary.json`
PowerShell	20 sequential API reliability requests and CSV/JSON summary creation	Executed; see `evidence/reliability-catalog-20.csv` and `evidence/reliability-summary.json`
Flutter `integration_test` / `flutter test`	Mobile login semantics and viewport-fit checks on Samsung SM-E556B, Android 16/API 36	Historical M02 passed; M01's source finder is now narrowed. Retest was blocked when phone disconnected; see `evidence/mobile/` and sibling mobile run log
ADB	Detect connected Android target and read model/OS information	Executed; serial `R9ZXA0206LN`
`dotnet run` / ASP.NET Core	Start the real API under test	Executed locally on port 5000
PostgreSQL	Database dependency for the API	Local PostgreSQL service was running; no test business data was changed

npm/Vite	Serve the real React application	Executed locally on port 3000
OWASP ZAP	Planned passive security scanning	Installer attempt was blocked by Windows security (flagged as malware/PUP); no scan report
Lighthouse/axe	Web accessibility and performance audit	Executed; axe had 0 detected violations in all three browsers; Lighthouse HTML/JSON saved
Firefox/WebKit	Cross-browser testing	Installed; login/browser and axe checks passed on both
Flutter profile tooling	Planned mobile performance/frame analysis	Not run; no Flutter performance result

CORS finding fix and retest (7 October 2026)

The original probe found that `https://untrusted.invalid` was echoed with `Access-Control-Allow-Credentials: true`. Replaced wildcard origin handling with `Cors:AllowedOrigins` configuration and `WithOrigins(...)`. Configured local origins are `localhost` and `127.0.0.1` on ports 3000 and 5173; deployments must set their actual frontend origins.

Build: `dotnet build .\Host\RentaTool.API\RentaTool.API.csproj --no-restore` succeeded with 0 warnings and 0 errors. OPTIONS retest to `/api/v1/equipment`: `http://127.0.0.1:3000` returned 204 with matching `Access-Control-Allow-Origin` and credentials headers; `https://untrusted.invalid` returned 204 with neither CORS permission header. Evidence: `evidence/cors-allowlist-retest.txt`. This verifies these CORS policy cases; it is not a replacement for ZAP or a broader security review.

API recovery test (7 October 2026)

Ran `nfr-testing/recovery/api_restart_recovery.ps1`. The script launched the local API, verified `/health` and `/api/v1/equipment` returned 200, stopped the process and observed the expected connection failure, then restarted it. `/health` returned 200 after **1.59 seconds** and the read-only equipment catalog returned 200. **All five recorded checks passed.** CSV/JSON results and process logs are in `evidence/recovery/`. This is API process recovery evidence only; browser/mobile retry behavior, recovery from an interrupted write, and mobile reconnection were not tested.

Executed commands and results

1. API (started with the locally configured Development environment):

```
   $env:ASPNETCORE_ENVIRONMENT='Development'; dotnet run --no-launch-profile --project
src/backend/Host/RentaTool.API/RentaTool.API.csproj --urls http://127.0.0.1:5000 --verbosity quiet
```

The initial sandboxed restore was denied access to NuGet (NU1301). It was retried with approved elevated execution and succeeded. API startup completed. `/health`, `/swagger/index.html`, and `/api/v1/equipment` returned HTTP 200; single-observation times were 6.88 ms, 2.20 ms, and 11.75 ms, respectively. The API startup/schema operations are the only database writes in this campaign.

2. Web server:

```
   npm.cmd run dev -- --host 127.0.0.1
```

Vite became available at `http://127.0.0.1:3000`.

3. Targeted NFR browser checks (real React page; no mocked API was needed by these page-load/layout checks):

```
   npx.cmd playwright test e2e/nonfunctional.spec.ts --reporter=list
```

Result: **5 passed**: mobile, tablet and desktop fit; login accessible names and keyboard focus; login page timing. The performance case now logs and attaches the measured navigation timing. The latest isolated retest recorded **230.60 ms DOMContentLoaded**; `src/web/evidence/nfr-performance-run.log` contains the output and the Playwright result has a

`navigation-timing.json` attachment. Screenshots are in `nfr-testing/evidence/web/`; original Playwright outputs remain in `src/web/test-results/`. The existing 5,000 ms guard is not a project SLA; 230.60 ms is one local observation.

Additional independent browser-engine check:

```
npx.cmd playwright test nfr-tests/browser-compat.spec.ts --config=playwright.nfr.config.ts --project=chromium
```

Result: **1 passed** in Chromium; screenshot saved in `nfr-testing/evidence/web/chromium-compat-login.png` and original in `src/web/nfr-test-results/`. Its first attempt timed out while Playwright waited for an external Google font during screenshot capture. The test was adjusted to abort external font requests (as the existing NFR spec already does), then rerun successfully. This was test harness setup, not an application defect.

After browser installation, the full login compatibility suite was run:

```
npx.cmd playwright test nfr-tests --config=playwright.nfr.config.ts --reporter=list
```

Result: **6 passed**: login compatibility and axe accessibility cases on Chromium, Firefox, and WebKit. Separate JSON and screenshot evidence for axe is saved in `evidence/axe-login-{chromium,firefox,webkit}.{json,png}`. The suite was rerun for evidence capture after output filenames were made browser-specific; the three accessibility cases passed again.

The main NFR page suite, including its added axe test, returned six passing test lines:

```
npx.cmd playwright test e2e/nonfunctional.spec.ts --reporter=list
```

Log: `src/web/evidence/nfr-web-playwright-final.log`. PowerShell wrapped a Node color warning as an error record, although Playwright printed **6 passed**; treat the six individual Playwright test outcomes as passed and the shell pipeline exit as non-clean.

4. Lighthouse production-preview audit:

```
npm.cmd run build
npm.cmd run preview -- --host 127.0.0.1 --port 4173
npm.cmd exec -- lighthouse http://127.0.0.1:4173/login --only-categories=accessibility,performance --
output=json,html --output-path=../../nfr-testing/evidence/lighthouse-login-final --chrome-path="$chrome" --
chrome-flags="--no-sandbox --ignore-certificate-errors"
```

Results: Lighthouse accessibility **1.00**, performance **0.86** on the local production preview. The earlier first audit identified the Show Password hit target as 22×28 px, below the 24×24 minimum used by Lighthouse's target-size audit (accessibility score 0.95). The button hit area was enlarged in `LoginPage.tsx`; the production retest reached accessibility 1.00. axe had zero automatically detected WCAG A/AA violations before and after. Lighthouse still reports performance opportunities such as unused JavaScript; no performance SLA is defined. HTML/JSON reports: `evidence/lighthouse-login-final.report.html` and `.report.json`.

5. API load test (read-only k6 profile):

```
k6 run .\nfr-testing\k6\read_only.js --summary-export .\nfr-testing\evidence\k6-summary.json
```

The profile sends GET requests to `/health` and `/api/v1/equipment` while ramping through 1, 3, and 5 VUs. The saved summary dated 6 October records **280 requests**, **280 checks passed**, **0 failed**, throughput **4.65 requests/s**, average response time **1.18 ms**, p95 **2.65 ms**, and maximum **28.31 ms**. No project latency SLA is defined, so this is a scenario result, not an SLA verdict. Evidence: `evidence/k6-summary.json`.

6. API security probes (equivalent calls were run individually with curl):

```
   curl.exe -sS -o NUL -w '%{http_code}' http://127.0.0.1:5000/api/v1/users
   curl.exe -sS -o NUL -w '%{http_code}' http://127.0.0.1:5000/api/v1/users/00000000-0000-0000-0000-000000000001
   curl.exe -sS -H 'Content-Type: application/json' --data-binary '@nfr-testing/evidence/invalid-login.json' -o
nfr-testing/evidence/invalid-login-response.json -w '%{http_code}' http://127.0.0.1:5000/api/v1/auth/login
   curl.exe -sS -D - -o NUL -X OPTIONS -H 'Origin: https://untrusted.invalid' -H 'Access-Control-Request-Method:
GET' -H 'Access-Control-Request-Headers: authorization' http://127.0.0.1:5000/api/v1/equipment
```

Results: both protected user reads and invalid login returned **401**. The original CORS probe returned **204** while echoing the arbitrary origin; this was later fixed and retested as documented above. See `evidence/security-probes.txt` for the original finding and `evidence/cors-allowlist-retest.txt` for the passing retest.

7. API reliability: a PowerShell loop sent 20 sequential read-only GETs to `/api/v1/equipment`, saving each status and elapsed time to `evidence/reliability-catalog-20.csv`. Latest run: 20/20 returned HTTP 200 with curl exit code 0. Summary: min 2.11 ms, mean 4.91 ms, p95 12.19 ms, max 38.13 ms, 0 transport/non-200 failures. The CSV stores time in seconds; the summary converts it to milliseconds. This is a small local repetition check, not a load/stress test and has no project SLA. To reproduce after starting the API:

```
   $timings = @(); for ($i = 1; $i -le 20; $i++) { $result = & curl.exe -sS -o NUL -w '%{http_code} %
{time_total}' --max-time 10 http://127.0.0.1:5000/api/v1/equipment; $parts = $result -split ' '; $timings +=
[pscustomobject]@{Iteration=$i; Status=[int]$parts[0]; Seconds=[double]$parts[1]} }; $timings | Export-Csv -
NoTypeInformation -Path nfr-testing/evidence/reliability-catalog-20.csv
```

8. Flutter device NFR checks on Samsung SM-E556B (Android 16/API 36):

```
   flutter test integration_test/auth_flow_test.dart -d R9ZXA0206LN --plain-name 'TC-NF-M0' 2>&1 | Tee-Object -
FilePath 'nfr_mobile_output.txt'
   flutter test integration_test/auth_flow_test.dart -d R9ZXA0206LN --plain-name 'TC-NF-M02' 2>&1 | Tee-Object -
FilePath 'nfr_mobile_m02_output.txt'
```

Historical TC-NF-M02 passed independently: login controls fit the 1080×2400 device viewport. Earlier TC-NF-M01 failed because its semantics finder returned both the visible label and the Semantics widget. The test source now explicitly finds the labeled `Semantics` widgets. A retest was attempted, but the device had disconnected: Flutter listed only Windows, Chrome, and Edge. Therefore the finder correction is **not yet retested on device**. Prior logs are in `evidence/mobile/`; the latest disconnected retest attempt is `RentaTool-mobile/nfr_mobile_output.txt`.

The existing screenshot-driver attempt failed because the project lacks a direct `package:test` dependency (`Could not find package test or file test:test`). Flutter test result logs are saved, but no screenshot was produced by that driver attempt.

9. Exploratory API stress test:

```
   k6 run .\nfr-testing\k6\stress_read_only.js --summary-export .\nfr-testing\evidence\k6-stress-summary.json
2>&1 | Tee-Object -FilePath .\nfr-testing\evidence\k6-stress-run.log
```

Stages ramped 5→10→20→30 configured VUs, then down to zero, for 70 seconds. The run recorded **3,780 requests**, **3,780 successful checks**, **0 failed requests**, peak observed **29 VUs**, average request duration **1.78 ms**, p95 **4.51 ms**, max **163.69 ms**, throughput **53.68 requests/s**. After ramp-down, `GET /health` returned 200 in 0.85 ms and `GET /api/v1/equipment` returned 200 in 3.63 ms. Evidence: `evidence/k6-stress-summary.json`, `evidence/k6-stress-run.log`, and `evidence/stress-post-run-probe.txt`. This is a moderate exploratory local stress profile, not a validated system capacity limit. No outage was introduced, so this does not establish recovery after a failure.

Defect found

DEF-NFR-001 — NFR-SEC-002 — FIXED AND RETESTED

- Original finding: API CORS permitted arbitrary origins with credentials.
- Severity: Medium (configuration weakness; browser-based cross-origin exposure depends on authenticated browser context and endpoint data sensitivity).

- Priority: High; deployment origins must be explicitly configured for each environment.
- Reproduction: send OPTIONS to `/api/v1/equipment` with `Origin: https://untrusted.invalid`, requested method GET, and Authorization header. See `evidence/security-probes.txt`.
- Expected: untrusted origin receives no credentialed CORS permission.
- Actual before fix: HTTP 204; arbitrary origin echoed with credentials allowed.
- Fix: replaced wildcard origin handling with configured `Cors:AllowedOrigins` and `WithOrigins(...)`.
- Retest: `http://127.0.0.1:3000` received matching allow-origin and credentials headers; `https://untrusted.invalid` received neither. See `evidence/cors-allowlist-retest.txt`.
- Deployment action: configure the exact production web/mobile origins for each environment before deployment.

No other defects were inferred from unavailable scans or test categories.

WEB-NFR-A11Y-002 — Fixed and retested: Lighthouse identified the Show Password button as an undersized touch target (22×28 px). The button now has a 24×24 px minimum hit area. The production-preview Lighthouse accessibility score increased from 0.95 to 1.00; axe reported zero automatic violations in Chromium, Firefox, and WebKit. Lighthouse performance score was 0.86; remaining performance opportunities are recorded in its HTML report.

BLOCKED checks and exact setup limits

- Load: the saved k6 summary records **280 requests** (140 health checks and 140 catalog checks); all 280 checks passed with **0 failed requests**. Throughput was 4.65 requests/s; average response time 1.18 ms, p95 2.65 ms, maximum 28.31 ms. No project-defined latency limit was found, so PASS means the scenario completed with successful endpoint checks, not that an SLA was met. Evidence: `evidence/k6-summary.json` (6 October 2026). Re-run from the RentaTool repository folder with the API running: `k6 run nfr-testing/k6/read_only.js --summary-export nfr-testing/evidence/k6-summary.json`.
- Stress: the read-only 5→10→20→30 VU exploratory profile completed without failed requests; no failure point was reached up to this level. This is not a capacity certification; expected production load and SLA remain undefined.
- ZAP: installation was attempted through winget, but Windows security blocked the official ZAP installer download as containing malware or potentially unwanted software. No workaround/bypass was attempted; no ZAP scan/report exists.
- Accessibility: axe and Lighthouse ran. The three-browser axe scans reported zero automatic violations; Lighthouse production preview scored accessibility 1.00 and performance 0.86. The axe contrast rule was marked incomplete for gradient backgrounds, so manual contrast review is still needed. These tools do not prove total WCAG conformance.
- Browser compatibility: Chromium, Firefox, and WebKit login checks passed. Coverage is only the login route and controls, not all business workflows.
- Flutter: historical M02 viewport check passed on the Samsung phone. M01's finder source was narrowed after its ambiguous result, but the phone disconnected before the retest; Flutter reported no matching device. Mobile frame performance, repeated/offline reliability, broader workflows and recovery remain untested. Android build output warned about SDK XML version compatibility, but prior filtered tests ran.
- Recovery: API stop/restart was tested; health recovered in 1.59 seconds and catalog returned 200. Client retry UX, interrupted-write data consistency and mobile reconnection remain untested.
- AI: Python runtime and live AI service were unavailable; AI subsystem NFR behavior is not covered.

Viva demonstration

1. Show `TEST_PLAN.md` and explain the scope limits: stress passed only at the tested level, ZAP remains blocked, Lighthouse/axe covered the login route, API recovery covered process restart, and several mobile NFR areas remain untested.
2. Open `src/web/test-results/` and show the Playwright screenshots for each viewport plus the login keyboard/name check.
3. Show `evidence/reliability-summary.json` and CSV; explain the 20 sequential read-only API calls and that this is reliability smoke evidence, not capacity evidence.

4. Re-run the two unauthenticated curl requests (both should return 401), then compare OPTIONS responses for an allowed origin and an untrusted origin. Show the CORS retest evidence and environment allowlist configuration.
5. Demonstrate `nfr-testing/k6/read_only.js` for load and `nfr-testing/k6/stress_read_only.js` for the separate staged exploratory stress run.
6. Open the Lighthouse HTML report and axe JSON/screenshots; explain the button target-size fix and the gradient contrast items axe could not determine automatically.
7. Show the ZAP installation block in the report; do not claim a ZAP scan. Reconnect an Android phone to rerun `TC-NF-M01` before presenting its corrected finder as passing.
8. Run `nfr-testing/recovery/api_restart_recovery.ps1` to demonstrate the API becoming unavailable and recovering after restart; explain the client retry/mobile recovery boundary.

NFR Test Plan

NFR test plan and outcomes

Test environment: local ASP.NET Development API (`127.0.0.1:5000`), PostgreSQL service, Vite React web (`127.0.0.1:3000`), and a Samsung SM-E556B running Android 16/API 36 for the historical mobile result. The phone disconnected before the latest mobile retest. No performance SLA was found in the project, so timing values are observations and are not pass/fail SLA judgments. BLOCKED/NOT RUN means no test result exists; the reason is stated in the row or detailed report.

Test ID	NFR Type	System Component	Feature/Scenario	Tool	Test Objective	Expected Result	Actual Result	Status	Evidence Location
NFR-PERF-001	Performance	Web	Login route first navigation at desktop viewport	Playwright Chromium	Measure DOMContentLoaded and check login renders	Timing is recorded; local 5,000 ms guard is not a project SLA	Latest retest: 230.60 ms DOMContentLoaded; login heading rendered; test passed	PASS (single local observation)	`src/web/evidence/nfr-performance-run.log`; Playwright `test-results/` attachment `navigation-timing.json`
NFR-PERF-002	Performance	API	Health and equipment catalog request	curl	Capture response time and response code	Endpoint responds; report observed latency without invented limit	health HTTP 200 in 6.88 ms; equipment HTTP 200 in 11.75 ms (single initial observations)	PASS (observed)	`evidence/api-probes.txt`
NFR-PERF-003	Performance	Flutter	Measure real-device frame/build performance	Flutter profile/integration tooling	Record frame timing/jank on target device	Actual frame metrics captured	Not run; no mobile performance instrumentation/profile capture was performed	NOT RUN	`FINAL_REPORT.md`
NFR-LOAD-001	Load	API/PostgreSQL	Gradual read-only health + equipment load, 1→3→5 VUs	k6	Record requests, latency distribution, errors, throughput	No data mutations; metrics reported; no project SLA available	Saved summary (`evidence/k6-summary.json`, 6 Oct): 280 requests; 280 checks passed; 0 failed; avg 1.18 ms, p95 2.65 ms, max 28.31 ms; 4.65 requests/s	PASS (scenario checks; no SLA)	`evidence/k6-summary.json`
NFR-STRESS-001	Stress	API/PostgreSQL	Read-only staged load at 5→10→20→30 VUs, then ramp down	k6	Observe response degradation/errors under exploratory load and check API after load stops	Actual load, errors, latency and post-load availability recorded; no project capacity SLA	3,780 requests; 3,780 checks passed; 0 failed; peak observed 29/30 VUs; avg 1.78 ms, p95 4.51 ms, max 163.69 ms; after ramp-down health/catalog returned 200. No failure point found up to this test level; no outage was induced	PASS (exploratory profile only; not capacity proof)	`evidence/k6-stress-summary.json`; `evidence/k6-stress-run.log`; `evidence/stress-post-run-probe.txt`
NFR-SEC-001	Security	API	Unauthenticated admin/user endpoints and invalid login	curl	Check authorization boundaries and bad credentials	Protected endpoints return 401; invalid login rejects; no token issued	Admin user list 401; user profile 401; invalid login 401	PASS (these probes only)	`evidence/security-probes.txt`; `evidence/invalid-login-response.json`

NFR-SEC-002	Security	API	CORS preflight from allowed local web origin and arbitrary origin	curl	Verify credentialed CORS is restricted to configured trusted origins	Allowed origin receives CORS headers; untrusted origin receives no CORS permission headers	Retest: `http://127.0.0.1:3000` returned 204 with matching allow-origin and credentials; `https://untrusted.invalid` returned 204 without allow-origin/allow-credentials headers	PASS (retested)	`evidence/cors-allowlist-retest.txt`; `src/backend/Host/RentaTool.API/Program.cs`; `appsettings.json`
NFR-SEC-003	Security	Web/API	Passive OWASP web/API scan	OWASP ZAP baseline	Identify scanner-reported passive findings on local app	Report findings by tool severity	BLOCKED: winget began ZAP 2.17 installation, but Windows security blocked the downloaded installer as malware or potentially unwanted software; scan not run	BLOCKED	`FINAL_REPORT.md`
NFR-USE-001	Usability	Web	Login form labels, password visibility control, keyboard focus; responsive viewports	Playwright Chromium	Check key login-page cues and layouts at 375/768/1440 px	Controls visible/labeled/focusable; no horizontal overflow	All 5 targeted tests passed	PASS	`src/web/test-results/`
NFR-USE-002	Usability	Flutter	Login field semantics and viewport fit	Flutter integration_test on Samsung SM-E556B, Android 16/API 36	Check labels, keyboard usability and screen fit	NFR assertions pass on real device	Historical M02 viewport-fit passed; M01 had an ambiguous finder and its test source is now narrowed to Semantics widgets. Retest attempted but blocked because the phone disconnected and Flutter found no device	PARTIAL (fix awaiting device retest)	`evidence/mobile/mobile-nfr-two-case-run.txt`; `RentaTool-mobile/nfr_mobile_output.txt`
NFR-A11Y-001	Accessibility	Web	Automated WCAG checks on login	Lighthouse + axe	Find common label, contrast, keyboard and semantic issues	Tool reports are saved; findings are triaged	axe: 0 violations across Chromium/Firefox/WebKit; Lighthouse production-preview accessibility score 1.00 after target-size fix; automated scans do not prove full conformance	PASS (automated scope)	`evidence/axe-login-*.json`; `evidence/lighthouse-login-final.report.html`; `evidence/lighthouse-login-final.report.json`
NFR-A11Y-002	Accessibility	Web	Login accessible names and keyboard focus	Playwright Chromium	Verify accessible naming/focus behavior	Named fields and keyboard focus work	Passed targeted browser assertion; not a full WCAG audit	PASS (limited)	`src/web/test-results/`
NFR-COMP-001	Compatibility	Web	Login page responsive layout in Chromium	Playwright Chromium	Check mobile/tablet/desktop viewport rendering	Controls fit viewport without horizontal overflow	3 viewport cases passed	PASS (single browser only)	`src/web/test-results/`
NFR-COMP-002	Compatibility	Web	Login route on Chromium, Firefox, WebKit	Playwright	Compare supported browser behavior	Each installed engine renders the route and records output	3 browser-rendering checks passed (Chromium, Firefox, WebKit)	PASS (tested login scope)	`src/web/nfr-test-results/`
NFR-COMP-003	Compatibility	Flutter	Login screen layout on physical Android phone	Flutter integration_test; Samsung SM-E556B, Android 16/API 36	Record actual device/OS and check screen behavior	Test result tied to actual device/OS	TC-NF-M02 viewport-fit passed on this device; coverage limited to this device and login screen	PASS (limited)	`evidence/mobile/mobile-viewport-fit-pass.txt`

NFR-COMP-004	Compatibility	Web	Login route/control rendering in Chromium	Playwright	Verify route and controls in installed engine	Controls visible; screenshot saved	1 Chromium browser-engine check passed	PASS (Chromium only)	`src/web/nfr-test-results/`
NFR-REL-001	Reliability	API	Repeat GET equipment catalog 20 times	PowerShell + curl	Check consistent response status and latency	Repeated requests remain valid; actual failures recorded	Latest run: 20/20 HTTP 200; 0 transport/non-200 failures; min 2.11 ms, mean 4.91 ms, p95 12.19 ms, max 38.13 ms	PASS (20 requests; no project SLA)	`evidence/reliability-catalog-20.csv`; `evidence/reliability-summary.json`
NFR-REL-002	Reliability	Flutter	Repeat login/API workflows and offline behavior	Flutter integration_test/device	Check consistency and app stability	Repeated real-device workflow results recorded	Not tested; no repeated/offline reliability case executed; phone currently disconnected	NOT RUN	`FINAL_REPORT.md`
NFR-REC-001	Recovery	API	Stop/restart local API process and probe health/catalog before and after	PowerShell recovery script + ASP.NET Core + PostgreSQL	Verify service becomes unavailable when stopped and recovers after restart	Unavailable state is detected; health and read-only catalog recover; elapsed recovery is recorded	API stopped: health probe returned expected connection failure; after restart health and catalog returned HTTP 200; health recovered in 1.59 seconds	PASS (API recovery scope; client retry/mobile recovery not tested)	`recovery/api_restart_recovery.ps1`; `evidence/recovery/api-restart-results.csv`; `evidence/recovery/api-restart-results.json`; API process logs

NFR Test Run Guide

SE3090 NFR run guide

This guide is for **non-functional testing only**. Run commands in PowerShell. From the repository root, use `C:\Users\ASUS\Desktop\Rental\RentaTool`. Start PostgreSQL before starting the API. The API uses its configured local database and runs its normal schema/seed checks on startup. Test calls below do not create business data.

Tools used and their purpose

Tool	Used for	Actual use/status
Playwright Test	React web usability, responsive layout, axe integration, browser compatibility and page timing	Chromium, Firefox, and WebKit used; login checks and axe tests passed
curl.exe	API response-time observations and security probes	Used against the live local API; HTTP statuses and timings were saved
k6	API load and exploratory stress testing	Load 1→3→5 and stress 5→10→20→30 virtual-user profiles executed; summaries saved
PowerShell	Repeat the API catalog GET 20 times, collect response codes/timings, export CSV	Used; all 20 calls returned HTTP 200
Flutter `integration_test` / `flutter test`	Mobile login usability and viewport-fit checks	Historical M02 passed; M01 finder was narrowed in source, but the latest retest was blocked because the Android device disconnected
Android Debug Bridge (ADB)	Detect and identify the attached Android device	Used to identify serial `R9ZXA0206LN`, model and Android version
ASP.NET Core `dotnet run`	Run the actual API under test	Used in Development at `http://127.0.0.1:5000`
PostgreSQL	Database used by the actual local API	Local service was running; API startup performed its normal schema/seed checks; test requests did not modify business data
Vite via npm	Serve the actual React application for Playwright	Used at `http://127.0.0.1:3000`; Playwright config can start it automatically
OWASP ZAP	Passive web/API security scan	Installation blocked by Windows security; no scan/report exists
Lighthouse and axe	Web accessibility and performance audits	Ran against login; axe zero violations, Lighthouse accessibility 1.00 and performance 0.86
Firefox and WebKit	Additional Playwright browser engines	Installed; login and axe checks passed
Flutter profile/performance tooling	Planned mobile frame/jank measurement	No performance measurement was run

Start services

Open PowerShell window 1, from the RentaTool repository root:

```
$env:ASPNETCORE_ENVIRONMENT = "Development"
dotnet run --no-launch-profile --project .\src\backend\Host\RentaTool.API\RentaTool.API.csproj --urls
http://127.0.0.1:5000
```

Leave it running for API, k6, security, and Flutter tests. Web Playwright starts Vite itself using its config, normally at `http://127.0.0.1:3000`.

Web usability, layout, accessibility smoke, performance observation

From `C:\Users\ASUS\Desktop\Rental\RentaTool\src\web`:

```
npx.cmd playwright test e2e/nonfunctional.spec.ts --reporter=list
```

This runs five real React login-page checks: mobile/tablet/desktop layouts, login accessible names and keyboard focus, and a page-render timing assertion. Previous result: **5 passed**. The timing assertion checks DOMContentLoaded under the test's existing 5-second budget; that budget is not a project SLA. The accessible-name check is a limited smoke check, not WCAG conformance.

The timing test logs the observed DOMContentLoaded duration and attaches `navigation-timing.json`. Latest isolated rerun: **230.60 ms**, test passed. Log: `src/web/evidence/nfr-performance-run.log`; this is one local observation, not an SLA.

Evidence:

- Screenshots: `RentaTool/nfr-testing/evidence/web/`
- Playwright raw output, trace, and screenshots: `RentaTool/src/web/test-results/`

Web compatibility

Run the installed Chromium check from `src\web`:

```
npx.cmd playwright test nfr-tests/browser-compat.spec.ts --config=playwright.nfr.config.ts --project=chromium
```

Previous result: **1 passed**. Screenshot: `RentaTool/nfr-testing/evidence/web/chromium-compat-login.png`.

The login route and its controls passed compatibility checks in all three engines:

```
npx.cmd playwright test nfr-tests --config=playwright.nfr.config.ts --reporter=list
```

Latest run: six login/browser and axe checks passed. HTML/raw output: `src/web/nfr-playwright-report/` and `src/web/nfr-test-results/`; per-engine axe JSON/screenshots: `nfr-testing/evidence/axe-login-{chromium,firefox,webkit}.{json,png}`. This covers the login page, not every business workflow.

API performance observation

With the API running, from any PowerShell window:

```
curl.exe -sS -o NUL -w "health: HTTP %{http_code}, %{time_total}s`n" http://127.0.0.1:5000/health
curl.exe -sS -o NUL -w "equipment: HTTP %{http_code}, %{time_total}s`n" http://127.0.0.1:5000/api/v1/equipment
```

Previously observed one-request results were health 200/6.88 ms and equipment 200/11.75 ms. These are single local observations, not an SLA. Saved initial measurements: `nfr-testing/evidence/api-probes.txt`.

API load test (k6)

Open a second PowerShell window at the RentaTool repository root, while the API is running:

```
k6 run .\nfr-testing\k6\read_only.js --summary-export .\nfr-testing\evidence\k6-summary.json
```

The prepared test ramps through 1, 3, then 5 virtual users and sends only GET requests to `/health` and `/api/v1/equipment`; it lasts about 60 seconds. The saved summary (`evidence/k6-summary.json`, dated 6 October) records **280 requests, 280 checks passed, 0 failed; 4.65 requests/s; avg 1.18 ms, p95 2.65 ms, max 28.31 ms**. No project latency limit was defined.

Evidence: `nfr-testing/evidence/k6-summary.json`. k6 console output appears in the terminal; to save a separate text log, add `2>&1 | Tee-Object -FilePath .\nfr-testing\evidence\k6-run.log` to the command.

Stress test

Run a separate read-only exploratory profile from the RentaTool root with the API running:

```
k6 run .\nfr-testing\k6\stress_read_only.js --summary-export .\nfr-testing\evidence\k6-stress-summary.json 2>&1
| Tee-Object -FilePath .\nfr-testing\evidence\k6-stress-run.log
```

Stages are 5→10→20→30 configured VUs then ramp down (70 seconds). Latest run: 3,780 requests/checks passed, 0 failed, 29 peak observed VUs, avg 1.78 ms, p95 4.51 ms, max 163.69 ms, throughput 53.68 requests/s. After ramp-down health/catalog returned HTTP 200; see `evidence/stress-post-run-probe.txt`. No errors or failure point were observed up to this test level. This is exploratory, not a capacity certification; no production load baseline or SLA is documented.

API security probes

With the API running, from the RentaTool repository root:

```
curl.exe -sS -o NUL -w "admin users without token: HTTP %{http_code}`n" http://127.0.0.1:5000/api/v1/users
curl.exe -sS -o NUL -w "user profile without token: HTTP %{http_code}`n"
http://127.0.0.1:5000/api/v1/users/00000000-0000-0000-0000-000000000001
curl.exe -sS -H "Content-Type: application/json" --data-binary "@.\nfr-testing\evidence\invalid-login.json" -o
.\nfr-testing\evidence\invalid-login-response.json -w "invalid login: HTTP %{http_code}`n"
http://127.0.0.1:5000/api/v1/auth/login
curl.exe -sS -D - -o NUL -X OPTIONS -H "Origin: https://untrusted.invalid" -H "Access-Control-Request-Method:
GET" -H "Access-Control-Request-Headers: authorization" http://127.0.0.1:5000/api/v1/equipment
```

Prior outcomes: protected user endpoints and invalid login returned 401. Run the fixed CORS check for an allowed local site and an untrusted origin:

```
curl.exe -sS -D - -o NUL -X OPTIONS -H "Origin: http://127.0.0.1:3000" -H "Access-Control-Request-Method: GET" -
H "Access-Control-Request-Headers: authorization" http://127.0.0.1:5000/api/v1/equipment
curl.exe -sS -D - -o NUL -X OPTIONS -H "Origin: https://untrusted.invalid" -H "Access-Control-Request-Method:
GET" -H "Access-Control-Request-Headers: authorization" http://127.0.0.1:5000/api/v1/equipment
```

The allowed origin should get matching `Access-Control-Allow-Origin` and credentials headers. The untrusted origin should get neither CORS permission header. Retest output is saved in `nfr-testing/evidence/cors-allowlist-retest.txt`; the original finding is preserved in `evidence/security-probes.txt`. Configure exact production origins in the deployment environment.

OWASP ZAP security scan

Not run. Winget began installing ZAP 2.17, but Windows security blocked the downloaded installer, labeling it malware or potentially unwanted software. I did not bypass that protection. No ZAP report exists. If resolved through your approved security software process, run a passive baseline scan against local web/API and save reports under `nfr-testing/evidence/`. Do not claim a ZAP result until the scan is executed.

API reliability repetition

With the API running, from `C:\Users\ASUS\Desktop\Rental\RentaTool`:

```
$timings = @()
for ($i = 1; $i -le 20; $i++) {
    $result = & curl.exe -sS -o NUL -w '%{http_code} %{time_total}' --max-time 10
http://127.0.0.1:5000/api/v1/equipment
    $parts = $result -split ' '
    $timings += [pscustomobject]@{ Iteration = $i; Status = [int]$parts[0]; Seconds = [double]$parts[1] }
}
$timings | Export-Csv -NoTypeInformation -Path .\nfr-testing\evidence\reliability-catalog-20.csv
```

This is a small sequential repeatability check, not a load test. Inspect each response status and duration in the CSV; the existing summary is `nfr-testing/evidence/reliability-summary.json`.

Accessibility audit

Dependencies and test harness are installed. Run axe against the login page in all three browsers:

```
cd C:\Users\ASUS\Desktop\Rental\RentaTool\src\web
npx.cmd playwright test nfr-tests/accessibility.spec.ts --config=playwright.nfr.config.ts --reporter=list
```

Latest run: zero automatically detectable WCAG A/AA violations in Chromium, Firefox and WebKit. To repeat Lighthouse, first build and run `npm.cmd run preview -- --host 127.0.0.1 --port 4173` in a separate terminal, then run:

```
$chrome = node -e "console.log(require('playwright').chromium.executablePath())"
npm.cmd exec -- lighthouse http://127.0.0.1:4173/login --only-categories=accessibility,performance --
output=json,html --output-path=../../nfr-testing/evidence/lighthouse-login-final --chrome-path="$chrome" --
chrome-flags="--no-sandbox --ignore-certificate-errors"
```

Latest production-preview scores: accessibility **1.00**, performance **0.86**. The password visibility button was enlarged after Lighthouse initially reported its hit area as 22×28 px; the retest reports no target-size failure. Evidence: `evidence/lighthouse-login-final.report.{html,json}`, `evidence/axe-login-{chromium,firefox,webkit}.{json,png}`. Axe marked several gradient contrast checks incomplete, so manual review remains useful. This audit covers `/login` only and is not a full conformance certificate.

Flutter unit, widget, integration, and mobile UI checks

The Flutter project is the sibling folder `C:\Users\ASUS\Desktop\Rental\RentaTool-mobile`. See `FLUTTER_TEST_REPORT.md` for the test cases and recorded outcomes. Its results are based on timestamped raw logs and the saved LCOV file; the older mobile `test_report.md` totals are superseded.

The historical Android target was Samsung SM-E556B, Android 16/API 36, ADB serial `R9ZXA0206LN`. The phone was disconnected during the latest M01 retest.

Connect an Android phone or emulator and identify it with `flutter devices`. Run commands from the Flutter project root:

```
flutter test --coverage
flutter test integration_test/auth_flow_test.dart -d <device-id> --plain-name 'TC-ID-'
flutter test integration_test/auth_flow_test.dart -d <device-id> --plain-name 'TC-NF-M01'
flutter test integration_test/auth_flow_test.dart -d <device-id> --plain-name 'TC-NF-M02'
flutter test integration_test/catalog_live_api_test.dart -d <device-id>
```

Latest saved statuses: the broad unit/widget output recorded 156 passes and 3 test-file load errors; the catalog API case passed; TC-NF-M02 passed historically on one Android device; the M01 device retest is blocked because no phone was connected. The latest saved full-app Flutter web E2E log records 4 passes and 1 login failure caused by a missing `flutter_secure_storage` plugin implementation. These are not full-suite passes.

API recovery test

The API process restart recovery scenario is implemented and was executed. To repeat it, first make sure no API is already using port 5000. The script starts and stops only the API process it launches; it refuses to run if the port is occupied. PostgreSQL must be running because the API performs its configured startup checks.

From the RentaTool repository root:

```
powershell.exe -NoProfile -ExecutionPolicy Bypass -File .\nfr-testing\recovery\api_restart_recovery.ps1
```

It probes `/health` and `/api/v1/equipment`, stops the local API process, confirms the health endpoint is unreachable, restarts the API, then probes health and catalog again. Latest run: all five checks passed; health recovered in 1.59 seconds. Results and API logs are written to `nfr-testing/evidence/recovery/`. This is API process recovery only; browser retry/user messaging, mobile reconnect, and data consistency across an interrupted write were not tested.

Flutter performance and reliability

Not run. The current Flutter NFR cases do not collect frame/jank metrics, repeated workflow behavior, or network outage/reconnect behavior. Flutter unit/widget tests are functional tests and are not substitutes for these NFR cases. No result or screenshot should be claimed for these categories until a device-targeted test has been implemented and executed.

Summary status

- **Executed:** web Playwright suite; Chromium/Firefox/WebKit login compatibility and axe; production Lighthouse; API timing and auth/CORS probes; k6 load and exploratory stress; API reliability loop; historical Flutter M02.
- **Fixed and retested:** login password-toggle target size; Lighthouse accessibility 1.00 and axe zero violations in the three engines; API CORS origin allowlist blocks untrusted origins.
- **Partial/blocked:** Flutter M01 source finder was narrowed but device retest is blocked by phone disconnect.
- **Not run/blocked:** OWASP ZAP (Windows security blocked installer), mobile performance/reliability/recovery and client retry behavior, live AI checks.

Full detailed results and the assignment test-plan matrix: `TEST_PLAN.md` and `FINAL_REPORT.md` in this folder.

Flutter Test Report

RentaTool Flutter Test Report

This report records Flutter test cases and the outcomes supported by saved logs. It is text-only; screenshots are not embedded in this testing guide.

Recorded results

Test area	Saved evidence	Recorded result
Unit and widget suite	`RentaTool-mobile/unit_test_output.txt` (6 October 2026)	**156 passed, 3 test files failed to load.** The failing files report `Undefined name 'main'` from the generated test listener. The latest saved broad run is not a full-suite pass.
Line coverage	`RentaTool-mobile/coverage/lcov.info`	**62.92%** (733 of 1,165 lines). This differs from the older mobile report's 64.46% figure.
Full-app Flutter web E2E	`RentaTool-mobile/e2e_output_fixed.txt` (3 October 2026)	**4 passed, 1 failed.** The login case failed with `MissingPluginException` for `flutter_secure_storage`; do not describe the suite as 5/5 passing.
Catalog API integration	`RentaTool-mobile/integration_test_output_flutter_xplat01.txt` (6 October 2026)	**1 test passed:** `XPLAT-01` read the shared seeded catalog.
Mobile login viewport	`RentaTool-mobile/nfr_mobile_m02_output.txt` (6 October 2026)	**TC-NF-M02 passed** on the Samsung SM-E556B running Android 16/API 36 at 1080 × 2400. This is one device and one login screen.
Mobile login accessibility/usability	`RentaTool-mobile/nfr_mobile_output.txt` (7 October 2026)	**Retest blocked:** no Android device was connected. The source finder was narrowed after an earlier ambiguous result, but the change has not been retested on device.

An earlier combined unit/widget log (`RentaTool-mobile/all_tests_combined_output.txt`, 4 October) records 229 passing tests. It is historical and does not replace the later 6 October run with three test-load errors. The older `RentaTool-mobile/test_report.md` contained superseded totals; this report uses the timestamped raw output and current LCOV file.

Flutter test cases

Identity integration cases

These nine cases are implemented in `RentaTool-mobile/integration_test/auth_flow_test.dart`. The current saved logs do not contain a case-by-case result for this suite, so each result is marked **not verified** rather than inferred from a screenshot or a different test run.

Case	Scenario	Expected result	Recorded result
TC-ID-01	Reject empty or malformed login details	Required-field and email validation is shown; login service is not called	Not verified in current saved logs

TC-ID-02	Reject invalid login credentials	Authentication error is shown and no token is stored	Not verified in current saved logs
TC-ID-03	Login and load profile trust data	Session is stored; catalog and authenticated profile display	Not verified in current saved logs
TC-ID-04	Validate required registration fields	Missing required values prevent registration	Not verified in current saved logs
TC-ID-05	Validate password confirmation and phone	Invalid values show validation feedback	Not verified in current saved logs
TC-ID-06	Handle a registration server error	Recoverable server error is displayed	Not verified in current saved logs
TC-ID-07	Register an owner	Selected owner role is submitted and catalog opens	Not verified in current saved logs
TC-ID-08	Reject an invalid Sri Lankan NIC	NIC validation prevents KYC submission	Not verified in current saved logs
TC-ID-09	Require the front KYC document image	Missing image prevents KYC submission	Not verified in current saved logs

Mobile non-functional cases

Case	Scenario	Expected result	Actual result	Status
TC-NF-M01	Login accessibility, field labels, keyboard use, and layout stability	Controls expose labels; keyboard use does not cause a layout exception	The finder was narrowed after an earlier ambiguous match, but the latest device attempt stopped because the Android device was unavailable	**PARTIAL — retest blocked**
TC-NF-M02	Login controls fit within the mobile viewport	Login fields and sign-in action remain visible within the viewport	Passed at 1080 × 2400 on Samsung SM-E556B, Android 16/API 36	**PASS — one device only**

Other Flutter flow cases

Test case	Scenario	Recorded result
XPLAT-01	Read the shared seeded catalog from the API	**PASS** in the saved 6 October run
E2E Catalog Flow	Catalog, search, add equipment, detail, inspection, and history	Test is implemented; no matching run result was found in the saved logs
E2E Booking Flow	Active bookings, booking details, QR handover, scanner, and map search	Test is implemented; no matching run result was found in the saved logs
E2E Escrow Flow	Escrow overview, claim details, and damage claim form	Test is implemented; no matching run result was found in the saved logs

Rerun guide

Run commands from `C:\Users\ASUS\Desktop\Rental\RentaTool-mobile`. Connect an Android phone or emulator first and check its ID with `flutter devices`.

Run unit and widget tests with coverage:

```
flutter test --coverage
```

Run the nine identity integration cases on Android:

```
flutter test integration_test/auth_flow_test.dart -d <device-id> --plain-name 'TC-ID-'
```

Run the mobile NFR cases separately:

```
flutter test integration_test/auth_flow_test.dart -d <device-id> --plain-name 'TC-NF-M01'
flutter test integration_test/auth_flow_test.dart -d <device-id> --plain-name 'TC-NF-M02'
```

Run the catalog API integration check with the API and its configured database available:

```
flutter test integration_test/catalog_live_api_test.dart -d <device-id>
```